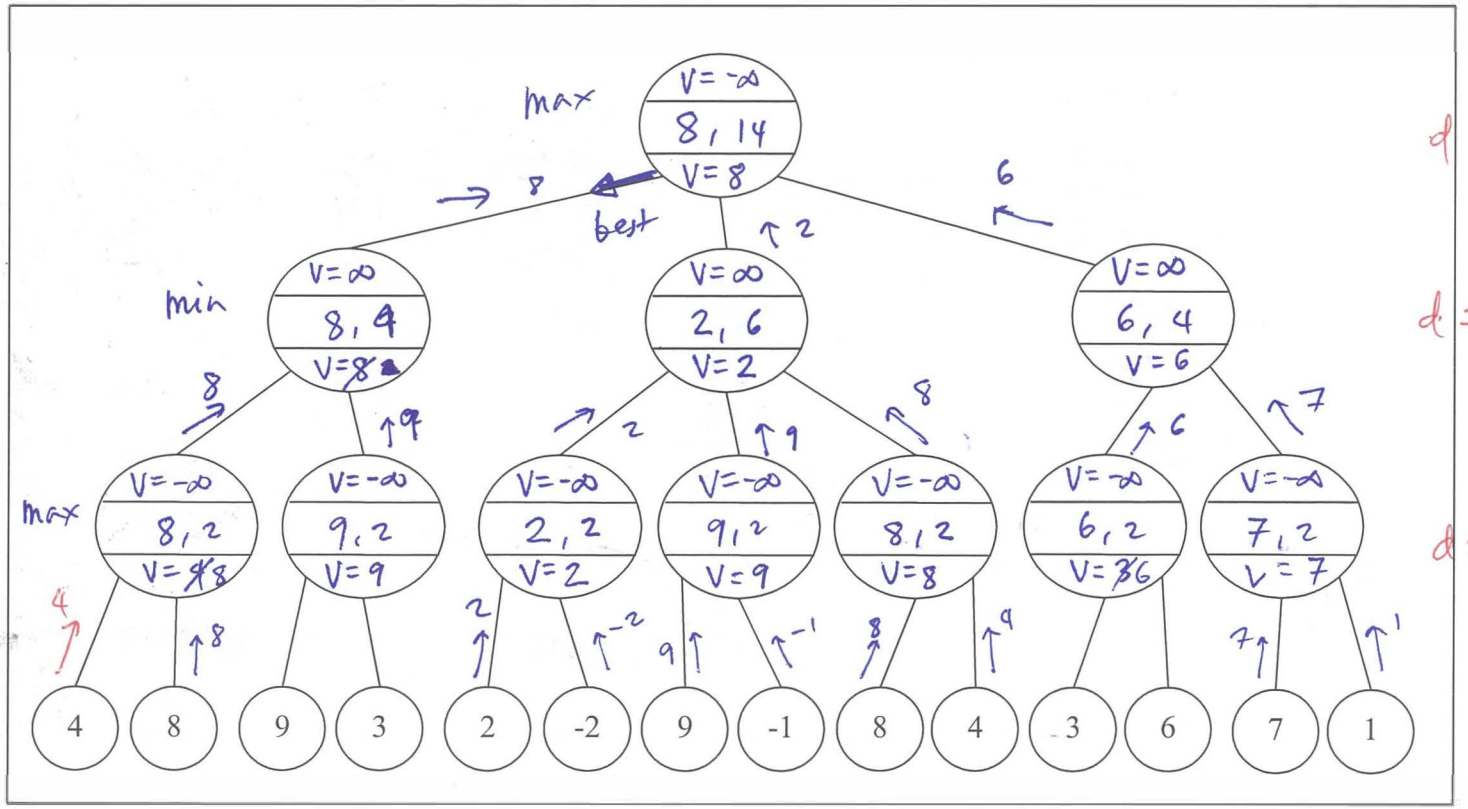
159302 Artificial Intelligence
Tutorial #8: GAMES: Alpha-Beta Pruning

Refer to the given game tree and simulate the MinMax algorithm. Show your complete answer by filling-in each of the three sections of the interior of the nodes with the required information.

MinMax Algorithm

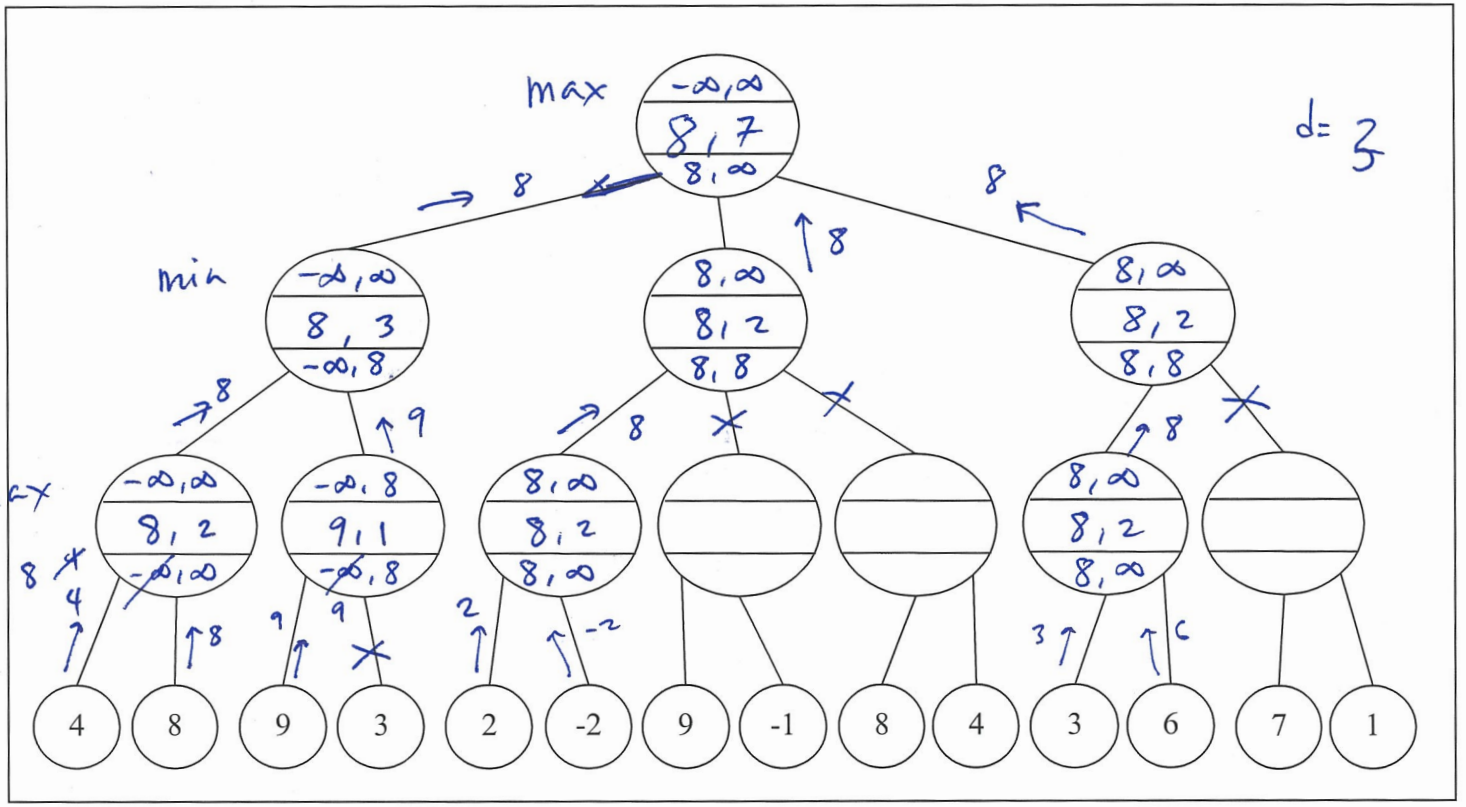


Source: MIT OpenCourseWare

159302 Artificial Intelligence
Tutorial #8: GAMES: Alpha-Beta Pruning

Refer to the given game tree and simulate the Alpha-Beta algorithm (consider nodes from left to right). Show your complete answer by filling-in each of the three sections of the interior of the nodes with the required information.

Alpha-Beta Pruning (left to right)

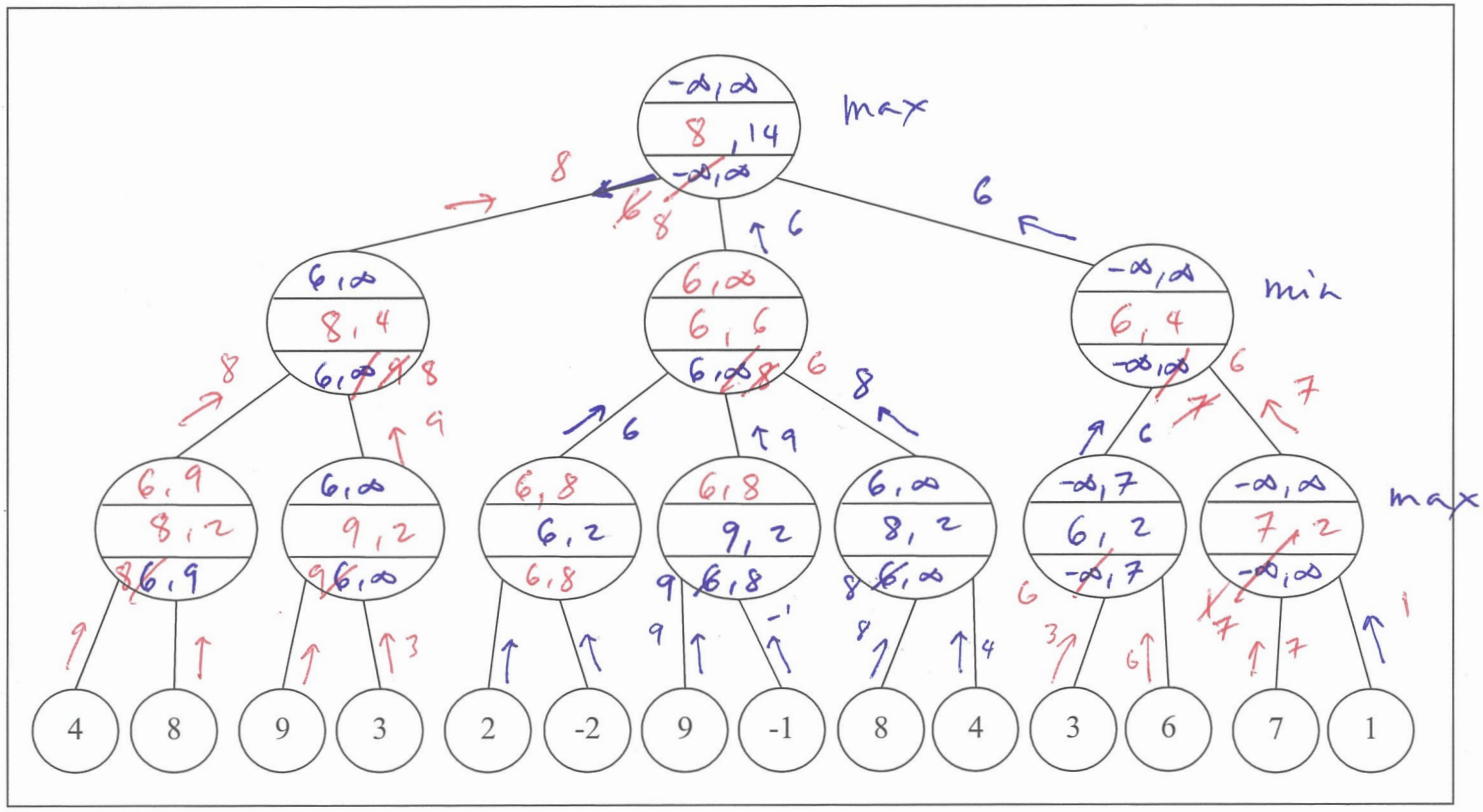


Source: MIT OpenCourseWare

159302 Artificial Intelligence
Tutorial #8: GAMES: Alpha-Beta Pruning

Refer to the given game tree and simulate the Alpha-Beta algorithm (consider nodes from right to left). Show your complete answer by filling-in each of the three sections of the interior of the nodes with the required information.

Alpha-Beta Pruning (right to left)



Source: MIT OpenCourseWare

159302 Artificial Intelligence
Tutorial #8: GAMES: Alpha-Beta Pruning

Min-MaX Algorithm

- *function MAX·VALUE(state, depth)*
  - *if (depth == 0) then return EVAL(state)*
  - *v = -∞*
  - *For each s in SUCCESSORS(state) do v = MAX(v, MIN·VALUE(s, depth – 1))*

end
return v

- *function MIN·VALUE (state, depth)*
  - *if (depth == 0) then return EVAL(state)*
  - *v = ∞*
  - *For each s in SUCCESSORS(state) do v = MIN(v, MAX·VALUE(s, depth – 1))*

end
return v

α-β (NegaMax form)

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is ***ALPHA·BETA****(state, -∞, ∞, MAX·DEPTH)*

*function **ALPHA·BETA**(state, α, β, depth)*
- *if (depth == 0) then return EVAL(state)*
- *For each **s** in SUCCESSORS(state) do*
  *α = MAX(α, -**ALPHA·BETA**(s, -β, -α, depth-1))*
  *If(α ≥ β) Then return **α** //cut-off*

end
return α

Alpha-Beta Pruning

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is *MAX·VALUE(state, -∞, ∞, MAX·DEPTH)*

*function **MAX·VALUE**(state, α, β, depth)*
- *if (depth == 0) then return EVAL(state)*
- *For each s in SUCCESSORS(state) do*
  α = MAX(α, MIN·VALUE(s, α, β, depth-1))
  If(α ≥ β) Then return α //cut-off

end
return α

*function **MIN·VALUE**(state, α, β, depth)*
- *if (depth == 0) then return EVAL(state)*
- *For each s in SUCCESSORS(state) do*
  β = MIN(β, MAX·VALUE(s, α, β, depth-1))
  If(β ≤ α) Then return β //cut-off

end
return β

Source: MIT OpenCourseWare

NegaMax tutorial

